

CSA0925 – PROGRAMMING WITH JAVA

CO3 – AT – 1

NAME : G SAI DEEPAK

REG : 192311432

Q1. ATM Withdrawal – Exception Handling Debugging

CODE:

```
// Online Java Compiler
import java.util.*;
public class Main{
public static void main(String arg[]){
Scanner sc=new Scanner(System.in);
try{
int b=sc.nextInt();
int a=sc.nextInt();
if(a==0){
throw new ArithmeticException("Withdrawal amount ");
}
```

```
if(a<0){
System.out.println("Invalid withdrawal amount");
}
else if(a>b){
System.out.println("Insufficient balance");
}
else{
int rb=b-a;
System.out.println("Withdrawal successful");
System.out.println("Remaining balance =" + rb);
}
}
catch(InputMismatchException e){
System.out.println("invalid input");
}
```

Q2. Hospital Registration – NullPointerException Debugging

CODE

```
import java.util.*;
public class Main{
public static void main(String args[]){
Scanner sc=new Scanner(System.in);
try{
String in=sc.nextLine();
```

```
String n;
if(in.equals("NULL")){
n=null;
}
else{
n=in;
}
int a=sc.nextInt();
if(n==null)
System.out.println("Patient name is unavailable");
else
System.out.println("Patient Name:"+ n.toUpperCase());
if(a>=18)
System.out.println("Adult Patient");
else
System.out.println("Minor Patient");
}
catch(InputMismatchException e){
System.out.println("Invalid age input");
}
catch(Exception e){
System.out.println("register failed");
}
finally{
System.out.println("registration completed");
```

```

}
}
}
catch(ArithmeticException e){
System.out.println(e.getMessage());
}
catch(Exception e){
System.out.println("Transaction Failed");
}
finally{
System.out.println("Transaction completed");
}
}
}
}

```

Q3. E-Commerce Product Search – Array Exception Debugging CODE

```

import java.util.*;

public class Main{

public static void main(String args[]){
Scanner sc=new Scanner(System.in);
String[] p={"Laptop","Mobile","Tablet","Headphone","Keyboard"};
try{
int pn=sc.nextInt();

```

```

if(pn<1|| pn>5){
System.out.println("Invalid product no:");
}
else
System.out.println("Selected product:"+ p[pn-1]);
}
catch(InputMismatchException e){
System.out.println("Ivalid input");
}
catch(Exception e){
System.out.println("Search failed");
}
finally{
System.out.println("Search completed");
}
}
}

```

Q4. Railway Reservation – Synchronization Debugging

CODE:

```

import java.util.*;

public class Main{

public static void main(String args[]){

```

```

Scanner sc=new Scanner(System.in);
String[] p={"Laptop","Mobile","Tablet","Headphone","Keyboard"};
try{
int pn=sc.nextInt();
if(pn<1|| pn>5){
System.out.println("Invalid product no:");
}import java.util.*;
public class Main{
public static void main (String args[]){
reservation r= new reservation();
customer c1=new customer(r,"Customer 1");
customer c2=new customer(r,"Customer 2");
c1.start();
c2.start();
}
}
class reservation{
int s=1;
synchronized void bookseat(String customer){
if(s>0){
System.out.println(customer+" is booking a seat");
try{
Thread.sleep(100);
}
catch(InterruptedException e){

```

```

System.out.println("thread interrupted");
}
s--;
System.out.println(customer+" Booking successful");
}
else
{
System.out.println(customer+" - no seat available");
}
class customer extends Thread{
reservation r;
String customername;
customer(reservation r,String customername){
this.r=r;
this.customername=customername;
}
public void run(){
r.bookseat(customername);
}
}
else
System.out.println("Selected product:"+ p[pn-1]);
}
catch(InputMismatchException e){
System.out.println("Invalid input");
}

```

```

}
catch(Exception e){
System.out.println("Search failed");
}
finally{
System.out.println("Search completed");
}
}
}
}

```

Q5. Bank Account – Race Condition Debugging

CODE:

```

public class Main{
public static void main(String args[])throws InterruptedException{
ba a=new ba();
atm atm1=new atm(a,7000);
atm atm2=new atm(a,6000);
atm1.setName("ATM-1");
atm2.setName("ATM-2");
atm1.start();
atm2.start();
atm1.join();
atm2.join();
System.out.println("final balance =" + a.balance);
}
}

```

```

}
class ba{
int balance=10000;
synchronized void withdraw(int amount){
if(balance>=amount){
System.out.println(Thread.currentThread().getName()+" withdrawal
"+amount);
try{
Thread.sleep(100);
}
catch(InterruptedException e){
System.out.println("Interrupted");
}
balance =balance-amount;
System.out.println(Thread.currentThread().getName()+" withdrawal
Successful");
}
else
System.out.println(Thread.currentThread().getName()+" Insufficient
Balance");
}
}
class atm extends Thread{
ba a;
int amount;
atm(ba a,int amount){

```

```
this.a=a;  
this.amount=amount;  
}  
public void run(){  
a.withdraw(amount);  
}  
}
```

THE END
